Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

CS 401/MCS 401 Lecture 2
Computer Algorithms I
Jan Verschelde, 17 June 2026

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

what is efficient?

We want algorithms to run efficiently.

For mathematical understanding, we need a definition

- independent of the platform on which the algorithm runs;
- independent of a specific implementation.

Every problem has an input size N.

Definition (1)

For a given problem, *the input size N* counts the number of integers any algorithm to solve this problem must receive on input.

the input size of the stable matching problem

Definition (1)

For a given problem, *the input size N* counts the number of integers any algorithm to solve this problem must receive on input.

For the stable matching problem with n employers and n applicants:

- every employer and applicant has a sequence of n preferences.
- The preferences for n employers are defined by n^2 integers.

Also n^2 define the preferences of n applicants.

So, $N = 2n^2$ for the stable matching problem.

measuring efficiency, worst case and brute force

Definition (2)

The largest possible running time an algorithm can have over all inputs of any given size N is *the worst-case running time* of the algorithm.

Example: for n employers and n applicants, the Gale-Shapley algorithm terminates after at most n^2 iterations. For input size $N = 2n^2$, the worst-case running time of this algorithm is bounded by $N/2$.

While worst-case may not accurately reflect the average running time, it is useful to compare with the running time of a brute-force algorithm.

For the stable matching problem of n employers and n applicants, there are $n!$ possible perfect matchings.
A brute force algorithm enumerates all possible $n!$ perfect matchings and to check each one to see if it is stable.

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

how does an algorithm scale?

If we double the input size, how does the running time increase?

Definition (3)

An algorithm *runs in polynomial time* for input size N if there are constants $c > 0$ and $d > 0$, independent of N, so that for any N, the running time is bounded by cN^d primitive computational steps.

Now we can define *efficient*.

Definition (4)

An algorithm is *efficient* if it has a polynomial running time.

examples of polynomial running times

Suppose our computer performs 1,000,000 instructions per second.
1 ms = 1 millisecond, 1 sec = 1 second, 1 min = 1 minute

N	N	$N\log_2(N)$	N^2	N^3
10	$<$ 1 ms	$<$ 1 ms	$<$ 1 ms	1 ms
30	$<$ 1 ms	$<$ 1 ms	$<$ 1 ms	27 ms
50	$<$ 1 ms	$<$ 1 ms	2 ms	125 ms
100	$<$ 1 ms	$<$ 1 ms	10 ms	1 sec
1,000	1 ms	9 ms	1 sec	18 min
10,000	10 ms	132 ms	2 min	12 days
100,000	100 ms	2 sec	3 hours	32 years
1,000,000	1 sec	20 sec	12 days	31,710 years

examples of exponential running times

Suppose our computer performs 1,000,000 instructions per second.
1 ms = 1 millisecond, 1 sec = 1 second, 1 min = 1 minute

N	1.1^N	1.5^N	2^N	$N!$
10	$<$ 1 ms	$<$ 1 ms	1 ms	3 sec
30	$<$ 1 ms	191 ms	18 min	10^{19} years
50	$<$ 1 ms	11 min	36 years	10^{51} years
100	13 ms	12,891 years	10^{17} years	∞
1,000	10^{28} years	10^{163} years	∞	∞
10,000	∞	∞	∞	∞

∞ is when an arithmetic overflow occurred when computing the time.

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

asymptotic growth, an example

Consider $p(N) = 2N^2 - N + 1$ and $q(N) = N^2 + 1000N + 12345$.

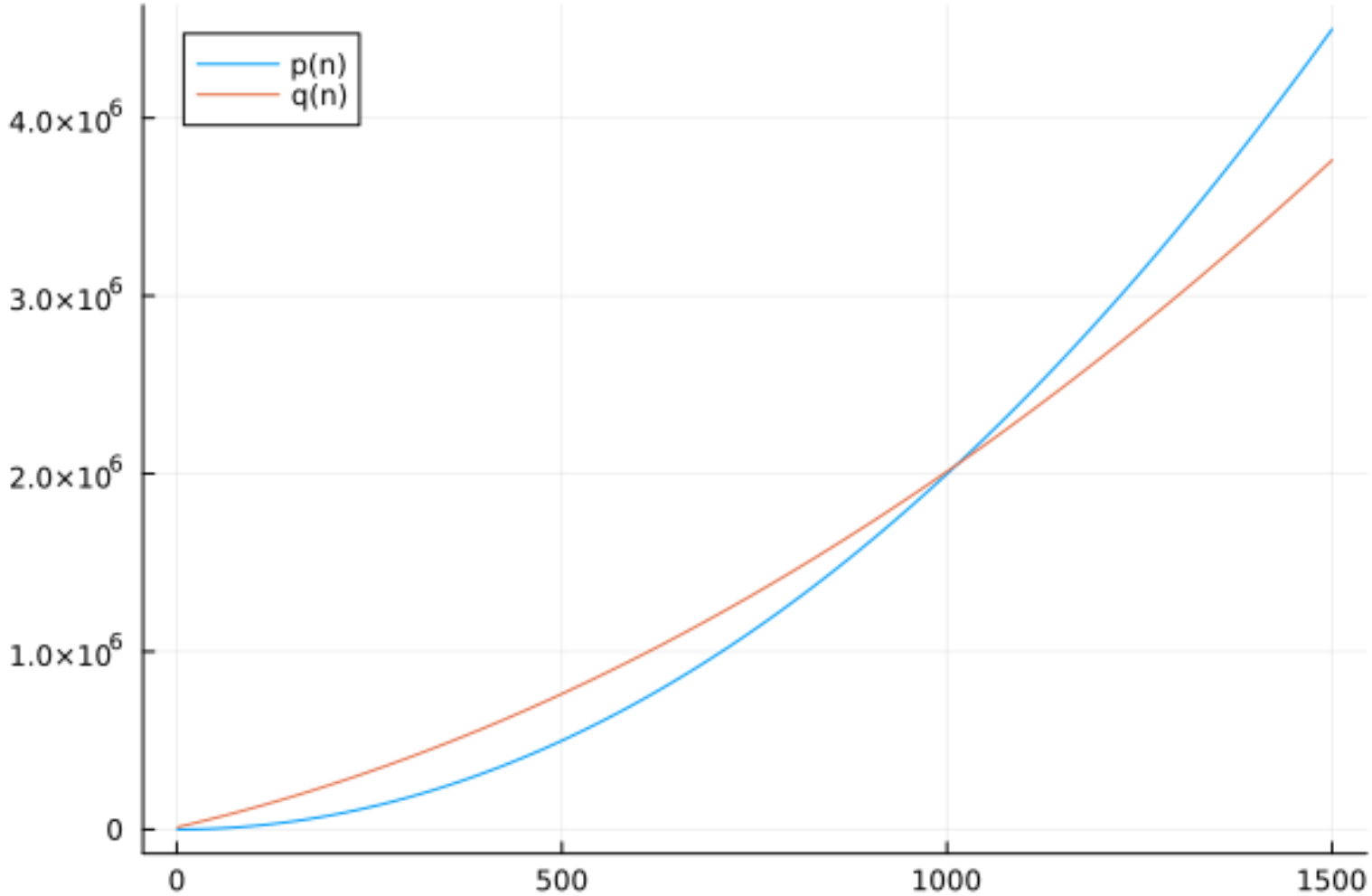
Which function grows faster? p or q?

N	$p(N)$	$q(N)$
1	2	13346
2	7	14349
4	29	16361
8	121	20409
16	497	28601
32	2017	45369
64	8129	80441
128	32641	156729
256	130817	333881

asymptotic growth, a plot

Consider $p(N) = 2N^2 - N + 1$ and $q(N) = N^2 + 1000N + 12345$.

Which function grows faster? p or q?



asymptotic growth, in the limit

Consider $p(N) = 2N^2 - N + 1$ and $q(N) = N^2 + 1000N + 12345$.

Which function grows faster? p or q?

Consider the limit of p over q as N goes to infinity:

$$\lim_{N \to \infty} \left(\frac{2N^2 - N + 1}{N^2 + 1000N + 12345} \right) = 2.$$

This means that for some $\epsilon > 0$, there exists a large constant N_0 so that

$$\text{for all } N > N_0 : \left| \frac{p(N)}{q(N)} - 2 \right| < \epsilon.$$

Alternatively: $p(N) \approx 2q(N)$ for sufficiently large N.

Asymptotically, only the leading term in a polynomial matters.

asymptotic upper bounds: big O, $O(\cdot)$

Both $p(N) = 2N^2 - N + 1$ and $q(N) = N^2 + 1000N + 12345$ are instances of quadratic growth.

We say that $p(N)$ is $O(N^2)$ and $q(N)$ is $O(N^2)$.

Let $T(N)$ be the worst case running time for input size N.

Definition (5)

For functions $T(N)$ and $f(N)$, $T(N)$ *is* $O(f(N))$ if there exists a constant $c > 0$, independent of N: $T(N) \leq cf(N)$, as $N \to \infty$, for all $N \geq N_0$, for a fixed constant value N_0.

Note that $O(\cdot)$ gives an upper bound, as N is also $O(N^2)$.

a first exercise

Exercise 1:

Consider two functions f and g such that $f(N)$ is $O(g(N))$.

Decide if $f(N)^2$ is $O(g(N)^2)$.

- If true, then give a proof.
- If false, give a counterexample.

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

asymptotic lower bounds: big Omega, $\Omega(\cdot)$

Let $T(N)$ be the worst case running time for input size N.

For example, $T(N) = N^2 + 2N + 3 \geq N^2$, for all N.

Definition (6)

For functions $T(N)$ and $f(N)$, $T(N)$ *is* $\Omega(f(N))$ if there exists a constant $c > 0$, independent of N: $T(N) \geq cf(N)$, as $N \to \infty$, for all $N \geq N_0$, for a fixed constant value N_0.

asymptotically tight bounds: big Theta, $\Theta(\cdot)$

For any $T(N) = c_2N^2 + c_1N + c_0$, $c_2 \neq 0$: $\lim_{N\to\infty}\left(\frac{T(N)}{N^2}\right) = c_2$,

so for some $\epsilon > 0$, for some sufficiently large N_0, for all $N \geq N_0$:

$$\left|\frac{T(N)}{N^2} - c_2\right| < \epsilon \quad \Leftrightarrow \quad -\epsilon < \frac{T(N)}{N^2} - c_2 < \epsilon$$

$$\Leftrightarrow \quad c_2 - \epsilon < \frac{T(N)}{N^2} < c_2 + \epsilon$$

$$\Leftrightarrow \quad (c_2 - \epsilon)N^2 < T(N) < (c_2 + \epsilon)N^2.$$

Definition (7)

$T(N)$ *is* $\Theta(f(N))$ if $T(N)$ *is* $\Omega(f(N))$ and $T(N)$ *is* $O(f(N))$.

Theorem (8)

Let $f(N)$ and $g(N)$ be functions so that $\lim_{N\to\infty} \frac{f(N)}{g(N)} = c > 0$,

for some constant c, independent of N, then $f(N)$ is $\Theta(g(N))$.

Proof. $\lim_{N\to\infty} \frac{f(N)}{g(N)} = c$ implies that for any $\epsilon > 0$,

there exists a constant N_0 so that for all $N \geq N_0$: $\left|\frac{f(N)}{g(N)} - c\right| < \epsilon$

$$\Leftrightarrow \quad -\epsilon < \frac{f(N)}{g(N)} - c < \epsilon \quad \Leftrightarrow \quad c - \epsilon < \frac{f(N)}{g(N)} < c + \epsilon$$
$$\Leftrightarrow \quad (c-\epsilon)g(N) < f(N) < (c+\epsilon)g(N).$$

By $(c-\epsilon)g(N) < f(N)$: $f(N)$ *is* $\Omega(g(N))$ and by $f(N) < (c+\epsilon)g(N)$: $f(N)$ *is* $O(g(N))$, so $f(N)$ *is* $\Theta(g(N))$. Q.E.D.

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

CS 401/MCS 401 Lecture 2
Computer Algorithms I
Jan Verschelde, 17 June 2026

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

examples of common running times

Let N be the size of the input.

- sublinear time $O(\log_2(N))$: binary search in sorted sequence
- linear time $O(N)$: find the maximum in an unsorted sequence
- $O(N \log_2(N))$ time: mergesort
- quadratic time: $O(N^2)$: smallest distance between points
- cubic time: $O(N^3)$: common elements in sequence of sets
- $O(N^k)$ time: search through all subsets of size k
- beyond polynomial time: brute force methods

sublinear time

Input: $S = x_1 \leq x_2 \leq \cdots \leq x_N$, a sorted sequence of numbers,
x some number.
Output: True if $x \in S$, False otherwise.

Binary search algorithm:

```
while #S > 0 do
    m := middle element in S
    if m = x then
        report True
    else if m > x then
        set S to its second half, after m
    else
        set S to its first half, before m
report False
```

Observe: the search space is cut in half in each iteration.

linear time

Input: $S = \{x_1, x_2, \ldots, x_N\}$, an unsorted set of numbers.
Output: $\max_{x \in S} x$.

Algorithm:

$m := -\infty$
for $x \in S$ do
  $m := \max(m, x)$.

One single loop through all elements of S, $\#S = N$.

$O(N \log_2(N))$ time

Input: $S = x_1, x_2, \ldots, x_N$, an unsorted sequence of numbers.
Output: the sorted sequence of N numbers in S.

Mergesort algorithm:

1. $S_1 :=$ Mergesort applied to the first half of the sequence S
2. $S_2 :=$ Mergesort applied to the second half of the sequence S
3. $\mathrm{merge}(S_1, S_2)$, merge the sorted S_1 and S_2 into the output

$T(N)$ is the time to sort N numbers.

1. Sorting half of the sequence takes time $T(N/2)$.
2. Merging two sequences of size $N/2$ takes time $O(N)$.

The recurrence $T(N) = 2T(N/2) + O(N)$ is $O(N \log_2(N))$.

a proof by induction

The recurrence $T(N) = 2T(N/2) + O(N)$ is $O(N \log_2(N))$.

Proof by induction on N:

1. The base case $N = 2$: $O(2)$ is $O(2 \log_2(2))$.
2. By induction, we assume it is true for all sizes $< N$, thus also for $N/2$: $T(N/2)$ is $O((N/2) \log_2(N/2))$.
3. We apply the induction: $T(N)$ is $2O((N/2) \log_2(N/2)) + O(N)$.

Using $O(N)$ is $O(N \log_2(N))$ and $\log_2(N/2) = \log_2(N) - \log_2(2)$ implies $T(N)$ is $O(N \log_2(N))$.

quadratic time

Given: $S = \{ (x, y) \mid x, y \in \mathbb{Z} \}$ points in the plane, $N = \#S$.

Wanted: minimum distance $d = \min_{(x,y),(x',y') \in S} \sqrt{(x - x')^2 + (y - y')^2}$.

Algorithm:

$d_{\min} := \infty$
for $(x, y) \in S$ do
  for $(x', y') \in S$, $(x', y') \neq (x, y)$ do
    $d_{\min} := \min(d_{\min}, \sqrt{(x - x')^2 + (y - y')^2})$.

Notice the nested, double loop:

1. The first loop is executed N times.
2. For the second loop we have $N - 1$ choices for (x', y').

So we have $N(N - 1)$ steps. Observe that the distance for $(x, y), (x', y')$ is the same as the distance for $(x', y'), (x, y)$, so solving this problem could be done in $\frac{1}{2}N(N - 1)$ steps, which is still $\Omega(N^2)$.

cubic time

Input: a sequence of N sets S_i, $i = 1, 2, \ldots, N$, $S_i \subseteq \{1, 2, \ldots, N\}$.
Output: is there a pair of sets which share no common element?

Algorithm:

```
for each set S_i do
    for each set S_j do
        disjoint := True
        for each p ∈ S_i do
            if p ∈ S_j then
                disjoint := False
        if disjoint then
            report (S_i, S_j).
```

Observe the three nested loops.

$O(N^k)$ time

Consider finding an independent set of size k in a graph.

In a graph, a set of nodes is independent
if no two nodes in the set are joined by an edge.

The algorithm runs through all subsets of size k of the nodes.

If the graph has N nodes, then the number of subsets of size k is

$$\binom{N}{k} = \frac{N(N-1)(N-2)\cdots(N-k+1)}{k(k-1)(k-2)\cdots 1} \leq \frac{N^k}{k!}.$$

Since k is a constant, the running time is $O(N^k)$.

beyond polynomial time

Consider a brute force method to solve the stable matching problem for n employers and n applicants:

1. Enumerate all possible perfect matchings.
2. For each perfect matching, check for stability.

Enumerate all possible perfect matchings:

1. For the first employer, there are n applicants to consider.
2. For the second employer, there are $n-1$ applicants to consider.
3. For the third employer, there are $n-2$ applicants to consider.
4. $\ldots$

There are $n(n-1)(n-2)\cdots = n!$ possibilities to consider.
The input size $N = 2n^2$, or $n = \sqrt{N/2}$.

The enumeration cost is then $\sqrt{N/2}!$, which is $O(N!)$.

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

transitivity of upper bounds

Theorem (9)

If $f(N)$ is $O(g(N))$ and $g(N)$ is $O(h(N))$, then $f(N)$ is $O(h(N))$.

Proof. If $f(N)$ *is* $O(g(N))$, then there is a constant $c > 0$ and some constant N_0 so that for all $N \geq N_0$: $f(N) \leq c\,g(N)$.

If $g(N)$ *is* $O(h(N))$, then there is a constant $d > 0$ and some constant M_0, so that for all $N \geq M_0$: $g(N) \leq d\,h(N)$.

So we have that for all $N \geq \max(N_0, M_0)$: $f(N) \leq c\,g(N) \leq c\,d\,h(N)$, which implies $f(N)$ *is* $O(h(N))$.

Q.E.D.

transitivity of lower bounds

Theorem (10)

If $f(N)$ is $\Omega(g(N))$ and $g(N)$ is $\Omega(h(N))$, then $f(N)$ is $\Omega(h(N))$.

Exercise 2: Give the proof for the above Theorem (10).

Corollary (11)

If $f(N)$ is $\Theta(g(N))$ and $g(N)$ is $\Theta(h(N))$, then $f(N)$ is $\Theta(h(N))$.

sums of two functions

Theorem (12)

If $f(N)$ is $O(h(N))$ and $g(N)$ is $O(h(N))$, then $f(N) + g(N)$ is $O(h(N))$.

Proof. If $f(N)$ *is* $O(h(N))$, then there is a constant $c > 0$ and some constant N_0 so that for all $N \geq N_0$: $f(N) \leq c\,h(N)$.

If $g(N)$ *is* $O(h(N))$, then there is a constant $d > 0$ and some constant M_0 so that for all $N \geq M_0$: $g(N) \leq d\,h(N)$.

So we have that for all $N \geq \max(N_0, M_0)$: $f(N) + g(N) \leq (c + d)h(N)$, which implies $f(N) + g(N)$ *is* $O(h(N))$.

Q.E.D.

sums of many functions

Theorem (13)

Let $f_1(N), f_2(N), \ldots, f_k(N)$ be a finite sequence of functions so that for all $i = 1, 2, \ldots, k$: $f_i(N)$ is $O(h(N))$.
Then $f_1(N) + f_2(N) + \cdots + f_k(N)$ is $O(h(N))$.

Exercise 3: Prove the above theorem.

asymptotic growth in a sum

Theorem (14)

Suppose $f(N)$ and $g(N)$ are two nonnegative functions such that g is $O(f)$. Then $f+g$ is $\Theta(f)$.

Proof. We have to show $f+g$ *is* $\Omega(f)$ and $f+g$ *is* $O(f)$:

1. For all $N \geq 0$, we have $f(N)+g(N) \geq f(N)$, so $f+g$ *is* $\Omega(f)$.
2. g *is* $O(f)$ and obviously f *is* $O(f)$, so $f+g$ *is* $O(f)$.

Thus, $f+g$ *is* $\Theta(f)$. Q.E.D.

Exercise 4: Prove the generalization of the above theorem for a finite sequence of functions, which are all $O(f)$.

For a finite sequence of functions, the sum grows as rapidly as the most rapidly growing function in the sequence.

Efficient Algorithms

1. Computational Tractability
   - defining efficiency
   - efficient algorithms
2. Asymptotic Order of Growth
   - asymptotic upper bounds
   - asymptotic lower and tight bounds
3. Examples, Properties and Bounds
   - examples of common running times
   - properties of asymptotic growth rates
   - asymptotic bounds for some common functions

polynomials

Theorem (15)

Let $f(N)$ be a polynomial of degree d, then f is $O(N^d)$.

Proof. The polynomial f is a sum of $c_i N^i$, for $i = 0, 1, \ldots, d$.
For all $i < d$ and for all $N \geq 1$: $N^i \leq N^d$.
Moreover, $c_i N^i \leq |c_i| N^i \leq |c_i| N^d$, for all $N \geq 1$, so $c_i N^i$ is $O(N^d)$.
The sum of functions that are $O(N^d)$ is also $O(N^d)$,
so f is $O(N^d)$. Q.E.D.

Exercise 5: Prove that a polynomial f of degree d is $\Theta(N^d)$.

$O(N^\pi)$ is also polynomial running time, even as N^π is not a polynomial.

logarithms

Recall: $\log_b(N)$ is the number x such that $b^x = N$.

Theorem (16)

For positive constants a and b: $\log_a(N)$ is $\Theta(\log_b(N))$.

Proof. Consider $\log_a(N) = \log_b(N)/\log_b(a)$.

Because $\log_b(a)$ is a constant, we have $\log_a(N)$ *is* $\Theta(\log_b(N))$.

Q.E.D.

Logarithms grow very slowly and the base of the logarithm does not matter, so we may just as well write $O(\log(N))$, omitting the base.

exponential growth

Consider $f(N) = r^N$, for $r > 1$.

We state a property as a theorem, because of its importance.

Theorem (17)

For every $r > 1$ and for every $d > 0$: N^d is $O(r^N)$.

Exponential growth is faster than any polynomial growth.

Consider:

$$\text{for } r > s > 1 : \lim_{N \to \infty} \left(\frac{r}{s}\right)^N = \infty.$$

Unlike logarithms, where the base does not matter,
the rate of growth of an exponential does matter.

For $r > s > 1$, s^N is *not* $\Theta(r^N)$.

the Gale-Shapley algorithm revisited

verifying whether the output is a perfect matching

How does the verification of the output of the Gale-Shapley algorithm compare against its running time?

Exercise 6:

Given is a list S of pairs (e, a) where e and a respectively belong to the sets of n employers and n applicants.

1. Design an algorithm to verify whether S defines a perfect matching.
2. Prove the correctness of your algorithm.
3. Express the cost of your algorithm as a function of n, using the big O and big Θ notation.

the Gale-Shapley algorithm revisited

verifying whether the output is a perfect stable matching

How does the verification of the output of the Gale-Shapley algorithm compare against its running time?

Exercise 7:

Given is a list S of pairs (e, a) where e and a respectively belong to the sets of n employers and n applicants.
In addition, the input contains the tables of preferences, both for the employers and the applicants.

1. Assuming S defines a perfect matching, design an algorithm to verify whether S defines a perfect stable matching.
2. Prove the correctness of your algorithm.
3. Express the cost of your algorithm as a function of n, using the big O and big Θ notation.